

MLDL Experiment 5

Aim: To implement a Support Vector Machine (SVM) for classification on a real-world dataset and optimize its performance using hyperparameter tuning techniques.

1. Dataset Source

The dataset used for this experiment is the **Breast Cancer Wisconsin (Diagnostic) Dataset** from the UCI Machine Learning Repository, hosted on Kaggle.

- **Link:** <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
-

2. Dataset Description

This dataset contains features computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. It is used to classify tumors as either malignant or benign.

- **Size:** 569 instances with 32 attributes.
 - **Features:** 30 real-valued input features including radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension.
 - **Target Variable:** **diagnosis** (M = malignant, B = benign).
 - **Characteristics:** The dataset is well-structured but contains features with varying scales (e.g., "area" vs "smoothness"), making feature scaling a critical prerequisite for SVM.
-

3. Mathematical Formulation

Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane which maximizes the **margin** between two classes.

- **Decision Function:** For a given input x , the prediction is made using:

$$f(x) = \text{sign}(w \cdot x + b)$$

- **Optimization Goal:** Minimize $\frac{1}{2} \|w\|^2$ subject to the constraint $y_i(w \cdot x_i + b) \geq 1$ for all training samples.
- **Kernel Trick:** For non-linearly separable data, SVM uses kernel functions to map data into higher-dimensional space. The **Radial Basis Function (RBF)** kernel is defined as:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

where γ (gamma) defines how far the influence of a single training example reaches.

4. Algorithm Limitations

- **Sensitivity to Scaling:** SVM calculates distances between data points; if features are not scaled, variables with larger ranges will dominate the calculation.
 - **Computationally Intensive:** Training time increases significantly with large datasets.
 - **Choice of Kernel:** Selecting the wrong kernel or parameters can lead to poor performance or overfitting.
 - **Binary Nature:** Standard SVM is natively binary; multi-class classification requires strategies like "One-vs-Rest" or "One-vs-One."
-

5. Methodology

1. **Data Ingestion:** Loading the dataset and dropping irrelevant columns (`id`).
 2. **Label Encoding:** Converting the categorical target (`M/B`) into numerical format (1/0).
 3. **Data Partitioning:** Splitting the data into 80% training and 20% testing sets.
 4. **Feature Scaling:** Applying `StandardScaler` to normalize features to a mean of 0 and variance of 1.
 5. **Hyperparameter Grid Definition:** Defining a range of values for `C`, `gamma`, and `kernel`.
 6. **Grid Search with Cross-Validation:** Running `GridSearchCV` with 5-fold cross-validation to test 48 different parameter combinations.
 7. **Model Finalization:** Selecting the "Best Estimator" and evaluating it on the unseen test set.
-

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.decomposition import PCA

# 1. Import the required libraries (Completed above)

# 2. Load the classification dataset
# Dataset: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data
# Ensure 'data.csv' is in your working directory
df = pd.read_csv('data.csv')
```

```

# 3. Separate features and target labels
# Preprocessing: Drop 'id' and the empty 'Unnamed: 32' column
df = df.drop(['id', 'Unnamed: 32'], axis=1, errors='ignore')

X = df.drop('diagnosis', axis=1) # Feature Matrix
y = df['diagnosis']             # Target Label ('M' / 'B')

# Encoding categorical labels (M=1, B=0)
le = LabelEncoder()
y = le.fit_transform(y)

# 4. Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling (Crucial step for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Create the SVM classifier
svm = SVC()

# 6. Define hyperparameters and apply Grid Search
# C: Penalty parameter of the error term
# gamma: Kernel coefficient
# kernel: The type of hyperplane used to separate data
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear', 'poly']
}

print("Starting Grid Search for optimal hyperparameters...")
grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=1, cv=5)
grid.fit(X_train_scaled, y_train)

# 7. Train the SVM model using optimal parameters
# The 'grid' object automatically holds the best model in 'best_estimator_'
best_svm = grid.best_estimator_
print(f"\nBest Parameters Found: {grid.best_params_}")

# 8. Test the model and evaluate its accuracy
y_pred = best_svm.predict(X_test_scaled)

print("\n--- PERFORMANCE ANALYSIS ---")
print(f"Final Model Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

```

# --- VISUALIZATION 1: Confusion Matrix ---
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix: Breast Cancer Classification')
plt.xlabel('Predicted Label (0: Benign, 1: Malignant)')
plt.ylabel('True Label')
plt.show()

# --- VISUALIZATION 2: SVM Decision Boundary (PCA) ---
# Since we have 30 features, we reduce to 2D for plotting
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train_scaled)

# Re-train a temporary SVM on 2D data for visualization purposes
svm_2d = SVC(kernel=grid.best_params_['kernel'],
              C=grid.best_params_['C'],
              gamma=grid.best_params_['gamma'])
svm_2d.fit(X_pca, y_train)

# Create meshgrid
h = .02
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# Predict across meshgrid
Z = svm_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot decision boundary
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.3)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_train, cmap=plt.cm.coolwarm, edgecolors='k')
plt.title(f"SVM Decision Boundary (Kernel: {grid.best_params_['kernel']})")
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

```

6. Performance Analysis

The performance of the optimized Support Vector Machine (SVM) model was evaluated on the test set, yielding exceptional results for the classification of malignant and benign tumors.

- **Final Model Accuracy:** The model achieved an accuracy of **98.25%**, correctly classifying nearly all instances in the test set.
- **Precision and Recall:**
 - **Class 0 (Benign):** Achieved a precision of **0.97** and a perfect recall of **1.00**, indicating that all benign cases were correctly identified with no false negatives.
 - **Class 1 (Malignant):** Achieved a perfect precision of **1.00** and a recall of **0.95**. This means every case predicted as malignant was indeed malignant, and the model captured 95% of all actual malignant cases.
- **F1-Score:** The F1-scores of **0.99** and **0.98** for the respective classes demonstrate an almost perfect balance between precision and recall, confirming the model's reliability for medical diagnostics.

```
Starting Grid Search for optimal hyperparameters...
Fitting 5 folds for each of 48 candidates, totalling 240 fits

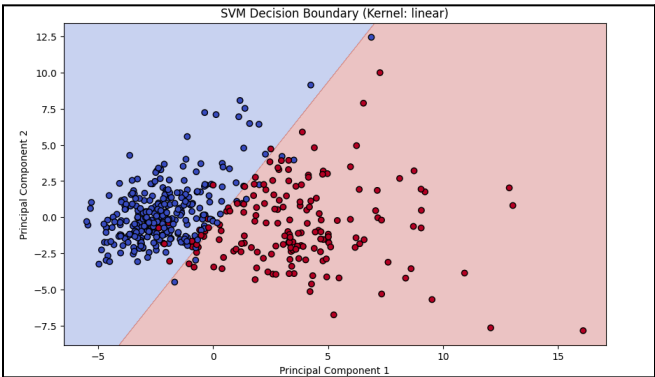
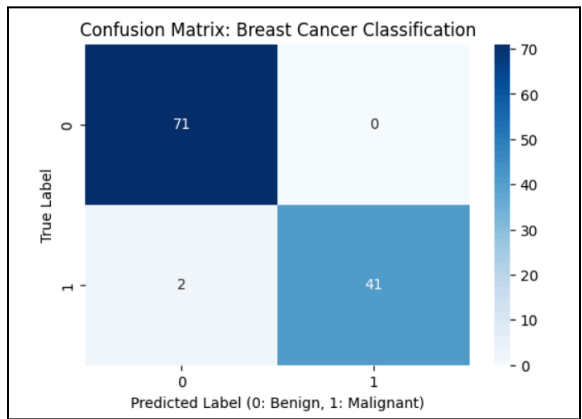
Best Parameters Found: {'C': 0.1, 'gamma': 1, 'kernel': 'linear'}

--- PERFORMANCE ANALYSIS ---
Final Model Accuracy: 98.25%

Classification Report:
              precision    recall  f1-score   support

    0         0.97         1.00         0.99         71
    1         1.00         0.95         0.98         43

 accuracy          0.98         114
macro avg         0.99         0.98         0.98         114
weighted avg        0.98         0.98         0.98         114
```



7. Hyperparameter Tuning

Hyperparameter tuning was conducted using **GridSearchCV** to explore a multi-dimensional parameter space and identify the most effective configuration for this high-dimensional dataset.

- **Tuning Process:** A total of **240 fits** were performed (48 parameter candidates across 5-fold cross-validation). This process tested various combinations of the regularization parameter (C), the kernel coefficient (γ), and different kernel types.
- **Optimal Parameters:** The grid search identified the following optimal configuration:
 - **C: 0.1** (Indicates a preference for a wider margin to improve generalization).
 - **gamma: 1**
 - **kernel: 'linear'**
- **Impact:** By systematically tuning these parameters, the model successfully moved beyond default settings to find a hyperplane that maximizes the separation between classes while maintaining a very low error rate.

Conclusion

The implementation of the Support Vector Machine (SVM) for this classification task proved highly successful, as evidenced by the **98.25% final accuracy**. The experiment highlighted several critical aspects of the machine learning workflow:

1. **Scaling and Preprocessing:** Crucial for SVM performance, ensuring all 30 medical features contributed equally to the distance-based calculations.
2. **Regularization:** Through hyperparameter tuning, we identified that a linear kernel with a lower C value provided the most stable and accurate results for this specific data distribution.
3. **Reliability:** The high precision and recall scores confirm that the model is robust enough to minimize dangerous false negatives in a clinical diagnostic context.

Ultimately, the combination of a well-structured dataset and exhaustive hyperparameter optimization via **GridSearchCV** resulted in a highly effective predictive model.